Loading the PIMA Indians Dataset:

In [3]:
```
import pandas as pd
import numpy as np

df = pd.read_csv(r"C:\Users\Rashid\Documents\AmeerahDiwanMATLABproject\diabetes\dia
df
```

Out[3]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction
0	6	148	72	35	0	33.6	0.627
1	1	85	66	29	0	26.6	0.351
2	8	183	64	0	0	23.3	0.672
3	1	89	66	23	94	28.1	0.167
4	0	137	40	35	168	43.1	2.288
...	...	...	...	...	...	...	...
763	10	101	76	48	180	32.9	0.171
764	2	122	70	27	0	36.8	0.340
765	5	121	72	23	112	26.2	0.245
766	1	126	60	0	0	30.1	0.349
767	1	93	70	31	0	30.4	0.315

768 rows × 9 columns

In [4]:
```
#Looking through the data and from the df.info() I can see that there are 768 rows
df.info()  #Tells me info about dataset.

#shows theres no 'nan' but Glucose and skin thickness have 0 and you cant ever have
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   Pregnancies               768 non-null    int64  
 1   Glucose                   768 non-null    int64  
 2   BloodPressure             768 non-null    int64  
 3   SkinThickness             768 non-null    int64  
 4   Insulin                   768 non-null    int64  
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64  
 8   Outcome                   768 non-null    int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

In [5]:
```
#because of glucose and other features containing 0's I want to the no. of 0's in d
zeros = (df == 0).sum()
zeros
#output shows there are 111 zeros in 1st feature which is acceptable because 0 preg
```

```
#Glucose having 5,BP,SkingThickness,Insulin and BMI cant have 0 since Glucose cant
#This tells me there are obviously errors and issues and perhaps missing values bec
```

Out[5]:
```
Pregnancies                 111
Glucose                       5
BloodPressure                35
SkinThickness               227
Insulin                     374
BMI                          11
DiabetesPedigreeFunction      0
Age                           0
Outcome                     500
dtype: int64
```

In [6]:
```
print(df.columns.tolist())
```

```
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'Di
abetesPedigreeFunction', 'Age', 'Outcome']
```

In [7]:
```
#These have bio impossible 0's and it doesnt make sense
zero_doesnt_make_sense = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]

#0's dont make sense like in Insulin then it should be cleaned.
columns_to_clean = []

for column in zero_doesnt_make_sense:  #basically is a loop that checks the feature
    if column in df.columns:
        columns_to_clean.append(column)

# Replace zero values with NaN to represent missing data
df[columns_to_clean] = df[columns_to_clean].replace(0, np.nan)   #treat as missing

df.isna().sum()  #shows missing no. in the features.

#now I have a better understanding of the missing no. amounts and if it is unrelist
```

Out[7]:
```
Pregnancies                   0
Glucose                       5
BloodPressure                35
SkinThickness               227
Insulin                     374
BMI                          11
DiabetesPedigreeFunction      0
Age                           0
Outcome                       0
dtype: int64
```

In [8]:
```
#Imputing as mentioned in comments this is next:
zero_doesnt_make_sense = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]
#using median as mentioned above median is more stable compared to using mean since
# Fill missing values using the median of each column
for column in zero_doesnt_make_sense:
```

```
    median_value = df[column].median()
    df[column] = df[column].fillna(median_value)
```

In [9]:
```
#Analysing and Understanding the Pima Indians Diabetes dataset:
#after imputing I want to look at the stats again to see the changes.
df.describe()

#I can see that the count is all 768 meaning that no missing values remain.
#Due to the outliers the mean would be pulled down so this wouldnt be as reliable d
#I can see that Insulin is very spread about since the standard deviation is large
#median gives middle amount so checks typicsl pregnancies is 1.
#Glucose levels range from 44 to 199 and the median is 117 and mean is 121.656250 s
#
#Since Glucose had a mean of 72.386719 meaning that the average Glucose level was t
```

Out[9]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPed
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	
mean	3.845052	121.656250	72.386719	29.108073	140.671875	32.455208	
std	3.369578	30.438286	12.096642	8.791221	86.383060	6.875177	
min	0.000000	44.000000	24.000000	7.000000	14.000000	18.200000	
25%	1.000000	99.750000	64.000000	25.000000	121.500000	27.500000	
50%	3.000000	117.000000	72.000000	29.000000	125.000000	32.300000	
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	

In [10]:
```
#Now since there is no clear distinction in the table of diabetics and non- diabeti
no_diabetes = df[df["Outcome"] == 0]
has_diabetes = df[df["Outcome"] == 1]

#I want to find out the average of each feature for diabetics and non- diabetics:

average_no_diabetes = no_diabetes.mean()
average_has_diabetes = has_diabetes.mean()

average_no_diabetes, average_has_diabetes

#1st part is for non-diabetics and 2 half is for diabetics.
#1st part tells me the averages for the features for non-diabetics:
#so-av pregnancies are 3, average Glucose level is 110.68, BP is 70.9, skinthicknes
#av diabetics are:
#around 5 pregnancies,Glucose=142.130597 etc.
#From comparing them I can see that Glucose in non-diabetics is of 110.68 as mentic
#pregnancies for those who were diabetic which was 4.86 so around 5 was higher thar
#Another comparison is that from comparing non-diabetics (30.81) and diabetics (35.
#non-diabetics average age was 31 compared to diabetes which was 37 suggesting most
#The genetic risk was higher for the diabetics since the diabetes pedigree score wa
#The different features show that there is a variety of different features that aff
```

Out[10]:
```
(Pregnancies                 3.298000
 Glucose                   110.682000
 BloodPressure              70.920000
 SkinThickness              27.726000
 Insulin                   127.792000
 BMI                        30.885600
 DiabetesPedigreeFunction    0.429734
 Age                        31.190000
 Outcome                     0.000000
 dtype: float64,
 Pregnancies                 4.865672
 Glucose                   142.130597
 BloodPressure              75.123134
 SkinThickness              31.686567
 Insulin                   164.701493
 BMI                        35.383582
 DiabetesPedigreeFunction    0.550500
 Age                        37.067164
 Outcome                     1.000000
 dtype: float64)
```

In [11]:
```
#Age Groups- splitting both to allow detailed analysis to be done.
age_patients = []

#creating loop to check the ages and to categorise them
for age in df["Age"]:
    if age < 30:
        age_patients.append("Under 30")
    elif age < 45:
        age_patients.append("30 to 44")
    else:
        age_patients.append("45 and above")

df["AgePatients"] = age_patients

df[["Age", "AgePatients"]].head()
```

Out[11]:

	Age	AgePatients
0	50	45 and above
1	31	30 to 44
2	32	30 to 44
3	21	Under 30
4	33	30 to 44

In [12]:
```
#BMI groups
bmi_patients = []

#loop for BMI

for bmi in df["BMI"]:
    if bmi < 25:
        bmi_patients.append("Normal")
    elif bmi < 30:
        bmi_patients.append("Overweight")
    else:
        bmi_patients.append("Obese")

df["BMIPatients"] = bmi_patients
```

```
df[["BMI", "BMIPatients"]].head()
```

Out[12]:

	BMI	BMIPatients
0	33.6	Obese
1	26.6	Overweight
2	23.3	Normal
3	28.1	Overweight
4	43.1	Obese

In [13]:
```
df.groupby("BMIPatients")["Outcome"].mean()
# what proportion of people in each BMI category have diabetes.
```

Out[13]:
```
BMIPatients
Normal        0.066038
Obese         0.457557
Overweight    0.223464
Name: Outcome, dtype: float64
```

In [14]:
```
# Create a binary flag for high glucose values
glucose_high = []

for glucose_score in df["Glucose"]:
    if glucose_score >= 140:
        glucose_high.append(1)
    else:
        glucose_high.append(0)

df["HighGlucose"] = glucose_high

# Check the new column
df[["Glucose", "HighGlucose"]].head()


# Select features that showed clear differences earlier
selected_columns = [
    "Glucose",
    "HighGlucose",
    "BMI",
    "Age",
    "DiabetesPedigreeFunction"
]

# Input data for the model
X = df[selected_columns]

# Target variable (diabetes or not)
y = df["Outcome"]
```

In [15]:
```
# Split the dataset into training and testing sets
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# I chose Logistic Regression because it is commonly used in healthcare
# and provides interpretable results for binary outcomes
from sklearn.linear_model import LogisticRegression

diabetes_LR = LogisticRegression(max_iter=1000)

# Train the model using the training data
diabetes_LR.fit(X_train, y_train)

# Make predictions on the test data (unseen data)
y_test_predictions = diabetes_LR.predict(X_test)

# Check how accurate the model is
from sklearn.metrics import accuracy_score

test_accuracy = accuracy_score(y_test, y_test_predictions)
test_accuracy
```

Out[15]:
```
0.7467532467532467
```

In [16]:
```
#confusion matrice-no of false positives and negatives
from sklearn.metrics import confusion_matrix

# Create a confusion matrix to see correct and incorrect predictions
confusion_matrice = confusion_matrix(y_test, y_test_predictions)
confusion_matrice
```

Out[16]:
```
array([[80, 19],
       [20, 35]], dtype=int64)
```

In [17]:
```
# Put the model coefficients into a table so they are easier to understand
coefficients_df = pd.DataFrame({
    "Influential Diabetic Features": selected_columns,
    "Coefficient": diabetes_LR.coef_[0]
})

coefficients_df

#from the table I can see that Glucose,BMI,Age and Diabetes pedigree function all h
#The HighGlucose coefficient was a negative but this was only because the Glucose l
```

Out[17]:

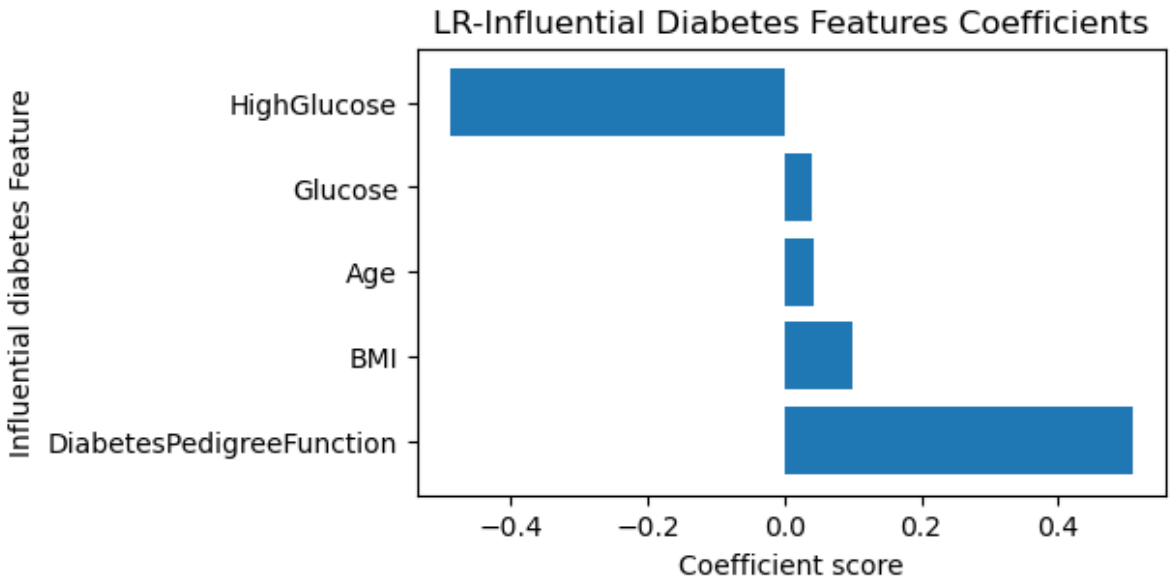
	Influential Diabetic Features	Coefficient
0	Glucose	0.041048
1	HighGlucose	-0.487776
2	BMI	0.098946
3	Age	0.042076
4	DiabetesPedigreeFunction	0.507734

In [18]:
```
#order coeff by importance
coeff_importance = coefficients_df.sort_values(
    by="Coefficient",
    ascending=False
)

coeff_importance
```

Out[18]:

	Influential Diabetic Features	Coefficient
4	DiabetesPedigreeFunction	0.507734
2	BMI	0.098946
3	Age	0.042076
0	Glucose	0.041048
1	HighGlucose	-0.487776

In [19]:
```
import matplotlib.pyplot as plt

# Create a simple bar chart of the coefficients
plt.figure(figsize=(5, 3))
plt.barh(
    coeff_importance["Influential Diabetic Features"],
    coeff_importance["Coefficient"]
)

plt.xlabel("Coefficient score")
plt.ylabel("Influential diabetes Feature")
plt.title("LR-Influential Diabetes Features Coefficients")
plt.show()

#Does genetic risk influence diabetes more compared to other features such as age,
#helps answer Do the trends and patterns reflected in the detailed analysis appear
#Since Glucose had a positive coefficient my earlier results were correct and that
#BMI also had a poitive coeffiecient supporting earlier results of high BMI linked
#Age-positive coefficient meaning diabetes is linked to older people more than your
#The diabetes pedigree function shows a strong positive coefficient meaning it matc
#Thus, the model did reflect the same results as produced earlier.
#the high glucose indicator as mentioned earlier was overlapping with Glucose and b
```



In [20]:
```
print("Logistic Regression Test Accuracy:", round(test_accuracy, 3))
```

```
Logistic Regression Test Accuracy: 0.747
```

In [21]:
```
# Extract values from the confusion matrix
true_negative = confusion_matrice[0][0]
false_positive = confusion_matrice[0][1]
false_negative = confusion_matrice[1][0]
true_positive = confusion_matrice[1][1]
```

```
true_negative, false_positive, false_negative, true_positive
#Helps me understand:
#How many non-diabetic cases were predicted correctly

#How many diabetic cases were predicted correctly

#Where the model makes mistakes
```

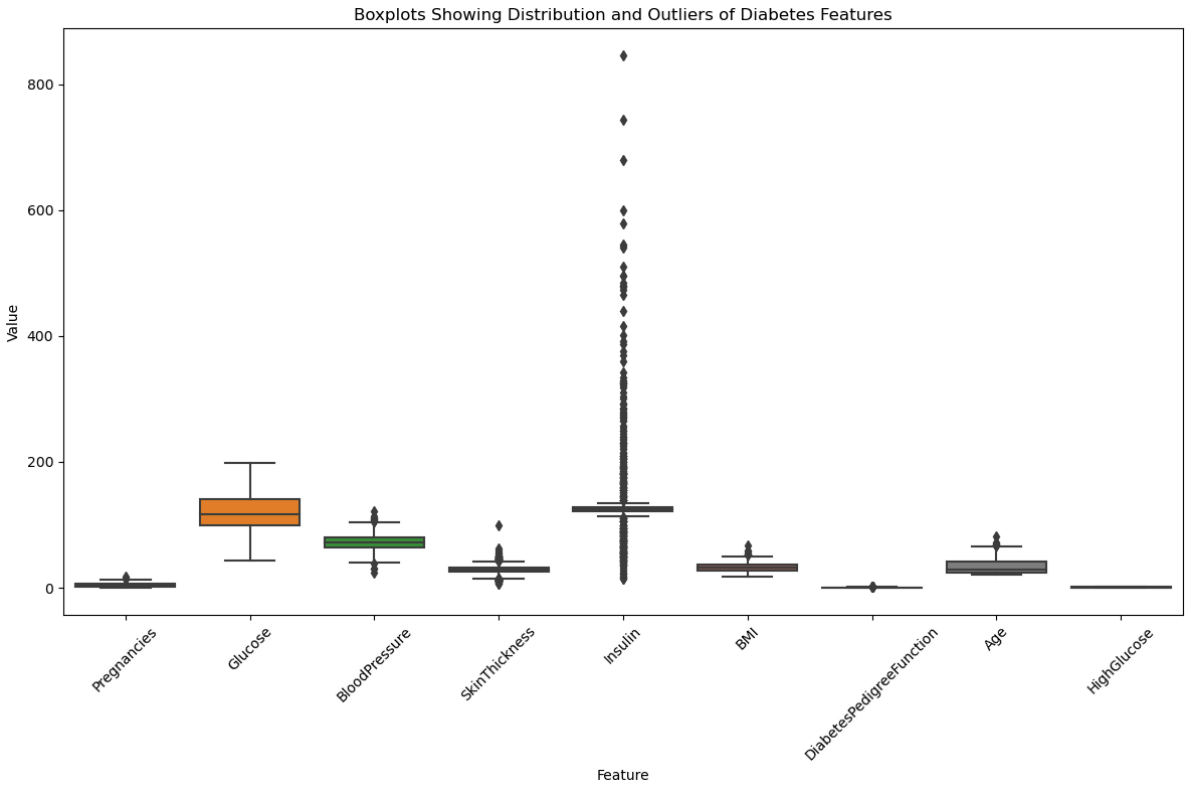
Out[21]:
```
(80, 19, 20, 35)
```

In [22]:
```
#used another metric since accuracy alone isnt enough and could be misleading becau
diabetes_recall = true_positive / (true_positive + false_negative)
diabetes_recall
```

Out[22]:
```
0.6363636363636364
```

Boxplot to show outliers in the PIMA dataset:

In [23]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# Select all numerical features from the dataset
diabetic_features = df.select_dtypes(include=["int64", "float64"]).columns

# Remove the target variable since it is not a clinical measurement
diabetic_features = diabetic_features.drop("Outcome")

# Reshape the data into a long format for seaborn
df_long = df[diabetic_features].melt(
    var_name="Feature",
    value_name="Value"
)

# Create boxplots to check distributions and outliers
plt.figure(figsize=(12, 8))
sns.boxplot(x="Feature", y="Value", data=df_long)

plt.xticks(rotation=45)
plt.title("Boxplots Showing Distribution and Outliers of Diabetes Features")
plt.xlabel("Feature")
plt.ylabel("Value")

plt.tight_layout()
plt.show()
```



In [24]:

```
import pandas as pd

# Store results here
outlier_tot = {}

# Go through each numeric column except the target
columns_choose = df.select_dtypes(include=["int64", "float64"]).columns

for feature in columns_choose:

    if feature == "Outcome":
        continue  # skip the target variable

    # Calculate quartiles
    q1 = df[feature].quantile(0.25)
    q3 = df[feature].quantile(0.75)

    # Interquartile range
    iqr_score = q3 - q1

    # Define outlier limits
    lower_bound = q1 - 1.5 * iqr_score
    upper_bound = q3 + 1.5 * iqr_score

    # Identify outliers
    outliers = df[(df[feature] < lower_bound) | (df[feature] > upper_bound)]

    # Percentage of outliers
    percentage_feature = (len(outliers) / len(df)) * 100

    outlier_tot[feature] = percentage_feature

# Convert to a table
outlier_percentage_feature_df = pd.DataFrame(
    list(outlier_tot.items()),
    columns=["Feature", "Outlier Percentage"]
)
```

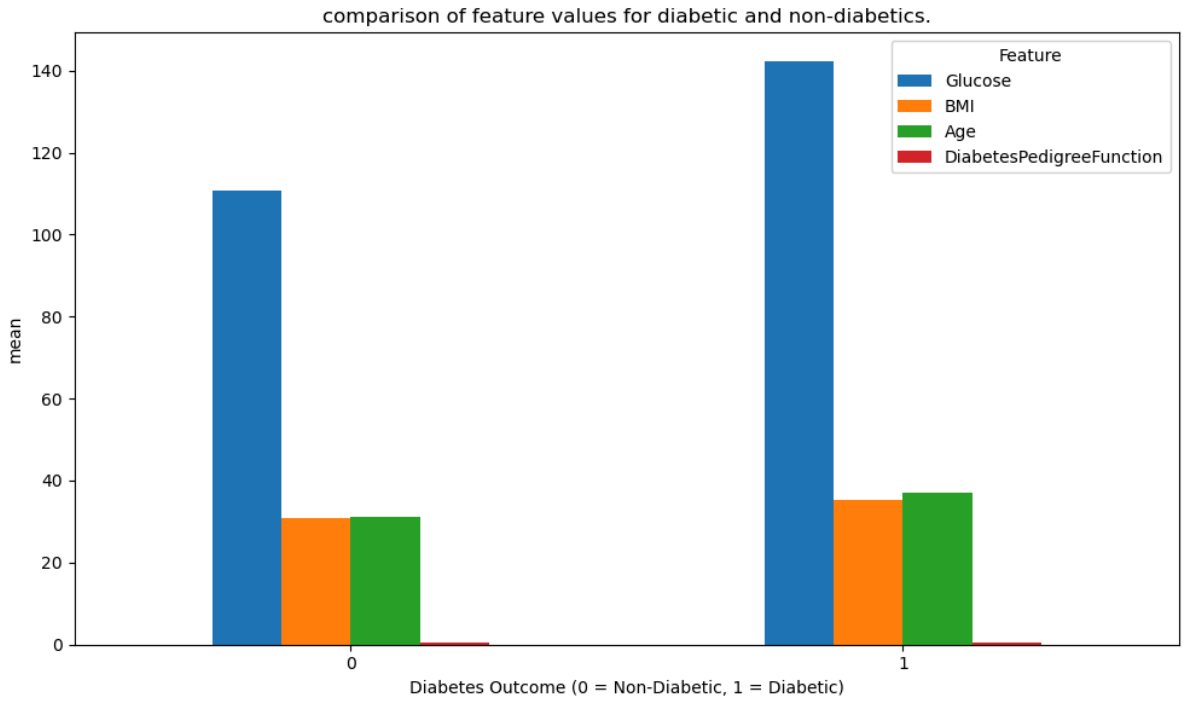
```
outlier_percentage_feature_df
```

Out[24]:

	Feature	Outlier Percentage
0	Pregnancies	0.520833
1	Glucose	0.000000
2	BloodPressure	1.822917
3	SkinThickness	11.328125
4	Insulin	45.052083
5	BMI	1.041667
6	DiabetesPedigreeFunction	3.776042
7	Age	1.171875
8	HighGlucose	0.000000

In [25]:

```
import matplotlib.pyplot as plt

# Select features to compare
compare_features = ["Glucose", "BMI", "Age", "DiabetesPedigreeFunction"]   #only ch

# Calculate mean values for each group
means_eachgroup = df.groupby("Outcome")[compare_features].mean()

# Create bar chart
means_eachgroup.plot(kind="bar", figsize=(10, 6))

plt.title("comparison of feature values for diabetic and non-diabetics.")
plt.xlabel("Diabetes Outcome (0 = Non-Diabetic, 1 = Diabetic)")
plt.ylabel("mean")
plt.xticks(rotation=0)
plt.legend(title="Feature")
plt.tight_layout()
plt.show()

#I used mean for comparisons to between the features since its better for comparing
```



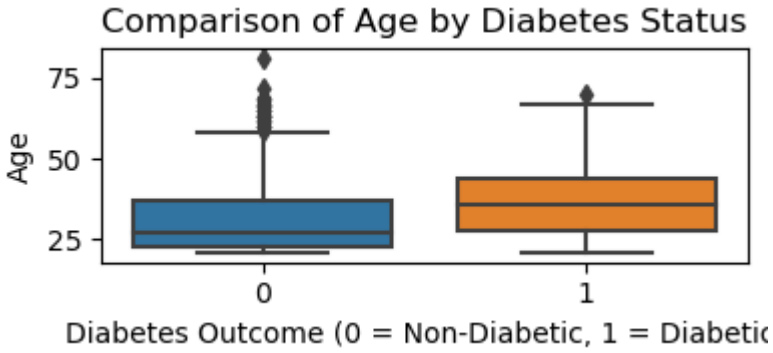
In [26]:
```
# Create a table showing the mean values used in the bar chart
means_eachgroup_table = means_eachgroup.copy()

# Rename the index so it is easier to read
means_eachgroup_table.index = ["Non-Diabetic", "Diabetic"]

# Display the table
means_eachgroup_table


#table shows the comparison of features for diabetics and non-diabetics and it show
#Does genetic risk influence diabetes using the Diabetes pedigree function link to
#Diabetic patients have higher average Glucose levels,this is expected since high b
#BMI is higher for those at an older average age suggesting a link between age and
#answers How do the influential features affecting diabetes like the health and dem
```

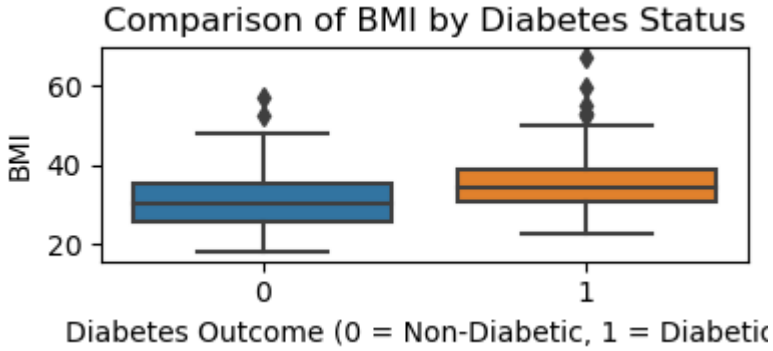
Out[26]:

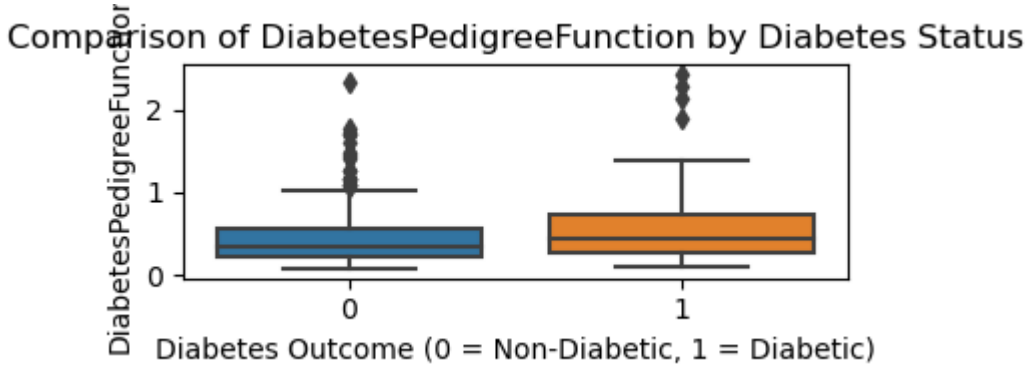
	Glucose	BMI	Age	DiabetesPedigreeFunction
Non-Diabetic	110.682000	30.885600	31.190000	0.429734
Diabetic	142.130597	35.383582	37.067164	0.550500

In [27]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# Select features needed for comparison
compare_features = ["Age", "BMI", "DiabetesPedigreeFunction"]

# Loop through each feature and create a boxplot
for feature in compare_features:

    plt.figure(figsize=(4, 2))
    sns.boxplot(x="Outcome", y=feature, data=df)

    plt.title(f"Comparison of {feature} by Diabetes Status")
    plt.xlabel("Diabetes Outcome (0 = Non-Diabetic, 1 = Diabetic)")
    plt.ylabel(feature)

    plt.tight_layout()
    plt.show()
```

```
#Boxplots show average age of people with diabetes are older than those without
#2nd shows that higher BMI is linked to increased diabetes risk.
#older people have higher BMI as mentioned so there may be a link between them
#3rd boxplot depicts that the average diabtic risk is higher for diabetics but
#since there is a small difference the difference is weaker compared to Age.
#answers how do the influential features affecting diabetes like the health and
```







In [28]:
```
# Create a table to summarise the boxplot features using the median
boxplot_data = df.groupby("Outcome")[compare_features].median()

# Rename the index for clearer interpretation
boxplot_data.index = ["Non-Diabetic", "Diabetic"]

# Display the table
boxplot_data


#table compares the different features and for those who are non-diabetic and diabe
#As mentioned there is a big difference between Age between diabetics and non-diabe
#The strongest differences were in Age and BMI than genetic risk so age and BMI are
#BMI has a smaller overlap so larger difference.
```

Out[28]:

	Age	BMI	DiabetesPedigreeFunction
Non-Diabetic	27.0	30.40	0.336
Diabetic	36.0	34.25	0.449

In [ ]:

In [ ]:

In [ ]: